
C247A Project Final Report

Beatrice Leung Justin Calderon Katie Moses
Valen Dunn
Department of Electrical and Computer Engineering
University of California, Los Angeles

Abstract

sEMG data captures time-series measurements of muscle activity from non-invasive sensors placed on the skin. We explore several neural network architectures which decode QWERTY keyboard strokes using sEMG user data. The architectures evaluated include CNN, RNN-LSTM, CNN+RNN-LSTM, RNN-GRU, RNN-GRU-RAU, and CNN+RNN-GRU models. Among these architectures, the RNN-GRU model achieved the best performance, obtaining a Test Character Error Rate (CER) of 13.5433. This paper outlines the structural differences in the above architectures and suggests the respective strengths and weaknesses of each model.

1 Introduction

This project investigates neural network architectures which minimize the Test CER value in a single subject. The CER, or Character Error Rate, analyzes the number of character substitutions, deletions, and insertions compared to the total number of characters. The Test CER metric evaluates architecture success in accurately decoding QWERTY keystrokes using sEMG user data. Neural network architectures are intentionally engineered to perform specialized functions and capture features within distinct types of data. This paper does not propose a universally superior model but rather emphasizes the importance of understanding the nature and structure of the data being analyzed and leveraging architectural choices to enhance model performance. The code can be found in the following Github repository: https://github.com/kmoses02/emg2qwerty_FinalProject

2 Methods

2.1 Dataset and Data Preprocessing

All experiments were conducted using the single-user configuration of the base *emg2qwerty* dataset, default training configuration, and provided preprocessing pipeline described in the next section. The base training setup includes three augmentation techniques designed to improve robustness to sensor placement and signal variability: *RandomBandRotation*, *SpecAugment*, and temporal jittering. *RandomBandRotation* simulates small rotations of electrode placement by permuting electrode band; *SpecAugment* randomly masks regions of the spectrogram in the time and frequency dimensions; and temporal jittering introduces small shifts in the signal timing. We conducted experiments with and without *RandomBandRotation* and *SpecAugment* for comparison.

2.2 Shared Model Pipeline

All architectures in this work share a common feature extraction and output pipeline. Input EMG signals are first converted into spectrogram representations. We apply 2D batch normalization and then a *MultiBandRotationInvariantMLP* module, which extracts spatial features across electrode

channels while maintaining robustness to small shifts in sensor placement while applying multiple linear layers.

The resulting feature sequence is passed to the architecture-specific encoder, which models temporal dependencies in the EMG signal. The encoder outputs are then mapped to character logits through a fully connected layer followed by a log-softmax activation. Training is performed using Connectionist Temporal Classification (CTC) loss, allowing the model to predict character sequences without explicit alignment between the EMG signal and the target text. All runs were done with 32 batches and ran for 50 epochs.

2.3 CNN Baseline Architecture

The baseline CNN model provided uses Time-Depth Separable (TDS) convolutional blocks Hanun et al. [2019]. Each block is a convolutional layer which filters only over time, followed by a Fully-Connected (FC) layer with residual connections. The baseline model stacks 4 of these blocks, followed by a fully connected layer to map extracted features to output characters.

2.4 LSTM Baseline Implementation

Another model we used was a bidirectional Long Short-Term Memory (LSTM) network. Standard recurrent neural networks (RNNs) suffer from the vanishing gradient problem, in which error signals decay exponentially as they propagate backwards through time, preventing the network from learning dependencies over long time horizons. This becomes problematic when dealing with EMG-based keystrokes in this project because signals can last for significant periods and need to be interpreted in the context of surrounding keystrokes.

Our model stacks multiple LSTM layers of

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (1)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (2)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (3)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad (4)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (5)$$

$$h_t = o_t * \tanh(C_t) \quad (6)$$

and processes the input both forwards and backwards to form a bidirectional LSTM. With this, the model is able to get context from information from both before and after the current moment being classified, NVIDIA Developer [2024]. This is better suited to the EMG data, as keystroke movements are shaped by finger movements, which are influenced by both past and future movements. The representations produced by the LSTM are first projected back to the original feature dimension by a linear layer, then normalized, and finally passed to a linear classifier with a log-softmax output. The model uses Connectionist Temporal Classification (CTC) loss to train on the A100 GPU.

2.5 CNN+RNN-LSTM Architecture

This model was originally implemented by preserving the baseline CNN model. The output features from the CNN's fully-connected layer were inputted into the Long Short-Term Memory (LSTM) RNN. Pytorch's LSTM architecture was used [2]. In this hybrid model, the CNN extracts features from the sEMG data and the LSTM captures temporal dependencies within the extracted features.

Next, a new CNN was implemented, modeled after the architecture proposed in iee [2020], exploring the use of a CNN+RNN-LSTM for sEMG-wrist kinematics analysis. This adapted CNN involved four consecutive convolutional layers followed by two FC layers. The second FC layer extracted feature vectors which were inputs for the LSTM architecture described above.

All tested architectures included a bidirectional LSTM and were trained on Colab's T4 GPU.

2.6 GRU Baseline Implementation

As a baseline recurrent architecture, we implemented a GRU-based sequence model using PyTorch's `nn.GRU` module. The model processes spectrogram features extracted from the EMG signals and

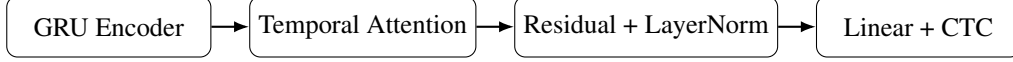


Figure 1: Simplified diagram of the GRU-RAU model.

predicts character sequences using a bidirectional GRU encoder. The GRU models used three layers with hidden sizes of: 256, 512, or 768. Learning rate of 0.001, 0.003 and 0.01 were used. One experiment was run with a drop-out of 0.3, compared to the base of 0.1. All experiments were trained on an A100 GPU, with training time per epoch typically ranging from approximately 18 to 27 seconds.

2.7 GRU-RAU Architecture

To improve the model’s ability to emphasize informative temporal segments of the EMG signal, we extended the baseline GRU architecture with a temporal attention mechanism inspired by the Recurrent Attention Unit (RAU) proposed by Zhong et al. [2018].

After the bidirectional GRU encoder produces the sequence of hidden representations $H = \{h_1, h_2, \dots, h_T\}$, an attention mechanism assigns a learned importance score to each timestep

$$e_t = W_a h_t + b_a \quad (7)$$

These scores are normalized using a softmax function

$$\alpha_t = \frac{\exp(e_t)}{\sum_{k=1}^T \exp(e_k)} \quad (8)$$

which produces attention weights representing the relative importance of each timestep. The GRU outputs are then reweighted

$$\tilde{h}_t = \alpha_t h_t \quad (9)$$

To preserve the original temporal representation while incorporating attention information, the attention-weighted representation is combined with the original GRU output through a residual connection followed by layer normalization

$$y_t = \text{LayerNorm}(h_t + \tilde{h}_t) \quad (10)$$

Unlike many attention mechanisms that compress the sequence into a single context vector, this design preserves the full temporal sequence so that the model remains compatible with the CTC decoding framework used for character prediction. Figure 5 illustrates the placement of the attention module between the GRU encoder and the final classification layer.

2.8 Inception-GRU Architecture

Perhaps capturing features across various timescales and considering them simultaneously can help an RNN find hidden patterns. Using the inception blocks proposed in GoogleNet Szegedy et al. [2014], this model applies multiple convolutional filters of different sizes to extract features across several time windows and channels. 1x1 filters are applied first to reduce the input to a lower dimensional encoding. These various features are then fed to a bidirectional GRU. Finally, a GRU with half the dimensions studies the high-level features. ReLU activations separate most layers. For simplicity, the number of GRU hidden states and 1x1 compression filters correspond to the same hyperparameter.

We experimented with 2 and 3 inception-GRU stacks. To mitigate potential vanishing gradients in the 3-stacks architecture, we again reference GoogLeNet by adding auxiliary loss from after the second inception-GRU stack. This is weighed less than the original loss, as the subnetwork does not need to learn the labels exactly. The architecture is visualized in figures 2 and 3.

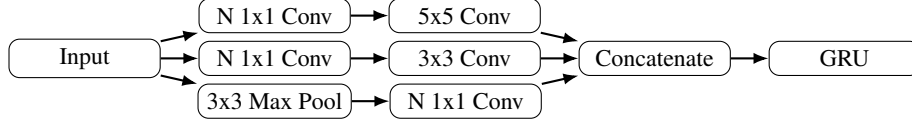


Figure 2: Simplified diagram of the Inception-GRU module.



Figure 3: Simplified diagram of the 3-stack Inception-GRU model with auxillary loss.

3 Results

3.1 LSTM Results

Table 1 shows the best performing runs of the LSTM architecture. A significant trend is overfitting: the test loss for the initial 3-layer model with a hidden layer of size 256 was 0.4, whereas the validation loss was significantly higher at 0.9. While the test error was sufficiently low, it was clear that the model still needed regularization, even with the current data augmentation.

While other models were tested, the 3-layer model consistently had the lowest loss and lowest CER. Regarding regularization, the clearest approach was to increase the dropout rate to reduce overfitting. We increased the dropout rate by 10 percent in each test until it began to reduce the model’s accuracy.

A dropout of 0.4 was found to be optimal, yielding a 15.26 CER for a bidirectional LSTM with 6.4 million trainable parameters.

Architecture	Dropout	Loss Gap	Test CER
LSTM	0	0.52	20.67
LSTM	0.1	0.48	18.98
LSTM	0.2	0.43	17.65
LSTM	0.3	0.38	16.66
LSTM	0.4	0.32	15.26
LSTM	0.5	0.36	16.65

Table 1: Performance of LSTM with different dropouts. Models were trained on a single-user dataset configuration

3.2 CNN+RNN-LSTM Results

The minimized Test CER value for this model was 22.89. The optimal dropout value was determined to be 0.1. Additionally, the number of LSTM layers and hidden state size were subject to experimentation. Performance was maximized with a hidden state size of 128 and 2 LSTM layers.

Using the tuned LSTM hyperparameters, the CNN constructed in accordance with lee [2020] was tested. The baseline CNN was replaced with the newly constructed CNN while the LSTM portion of the model remained unchanged. The Test CER value using newCNN+RNN-LSTM was 26.11, indicating better performance from the baseline CNN model (Test CER = 22.89).

Run #	Architecture	Hidden Size	Layers	Dropout	Val CER	Test CER
1	baselineCNN-LSTM	128	2	0.0	29.5968	27.6421
2	baselineCNN-LSTM	128	2	0.1	21.0013	22.8874
3	baselineCNN-LSTM	128	2	0.2	25.6978	25.7834
4	baselineCNN-LSTM	128	3	0.1	22.2862	23.2548
5	baselineCNN-LSTM	64	2	0.1	22.6628	24.7677
6	newCNN-LSTM	128	2	0.1	26.7390	26.1076

Table 2: CNN+LSTM run summary.

3.3 GRU Results

Table 3 summarizes the best-performing GRU configurations. The best-performing GRU configuration used a hidden size of 256 with three layers, achieving the lowest Test CER, 13.54, while using significantly fewer parameters than larger models. Smaller models were often more parameter-efficient while achieving comparable or better performance.

One clear trend observed in our experiments was the importance of the data augmentation pipeline. For runs performed without augmentation, the Test CER was consistently worse than the Validation CER, indicating reduced generalization to unseen sequences.

We also explored the effect of model size and depth. Increasing the hidden size from 256 to 512 or 768 did not consistently improve performance. In several cases, models with hidden size 768 achieved competitive Validation CER but produced worse Test CER. Similarly, increasing the number of layers from two to three did not produce large performance differences across configurations.

Three additional observations were noted during experimentation. Increasing the learning rate beyond 0.001 resulted in worse performance across runs. In addition, the configuration trained without bidirectional recurrence produced noticeably poorer results than comparable bidirectional models. When a Dropout rate of 0.3 was used, the resulting test CER was significantly poorer, 34.1042, compared to 20.4884, using three hidden layers of size 768. When learning rate was increased from 0.001 to 0.01, results were much worse, including a test CER of 93.84.

3.4 GRU-RAU Results

Table 3 also includes the best-performing GRU-RAU configurations. The best-performing GRU-RAU configuration used a hidden size of 256 with two layers, achieving the lowest Test CER, 17.7221.

The GRU-RAU architecture produced generally competitive results across most configurations, but it did not consistently outperform the baseline GRU when comparing models with similar hidden sizes and layer counts, suggesting that the attention mechanism did not provide a consistent advantage for the single-user decoding setting evaluated in this work.

Architecture	Hidden Size	Layers	Augmentation	Test CER	Params
GRU	256	3	Yes	13.54	4.8M
GRU	512	3	Yes	19.49	14.7M
GRU	768	3	Yes	20.49	30M
GRU-RAU	256	3	Yes	18.80	4.8M
GRU-RAU	512	3	Yes	23.90	14.6M
GRU-RAU	768	3	Yes	28.96	30M

Table 3: Best-performing runs for GRU and GRU-RAU architectures after 50 training epochs. Models were trained on a single-user dataset configuration.

3.5 Inception-GRU

Table 4 shows experimental results. Both inception-GRU architectures performed similarly for accuracy. Despite adding more layers, the 3-stack network had a negligible increase in the number of parameters. Test CER was higher than validation CER, suggesting overfitting for the 3-stack network especially.

The 3-stack model maintained higher loss by some constant term over training, but they followed the same trajectory. Both CTC loss and CER score dropped noticeably around epoch 25 for both architectures; however the 3-stack model showed a peculiar spike in validation CER during this time. It also had slightly more variable training loss since epoch 40. Perhaps the submodels in this variant are learning against each other, since both use CTC loss against the ground labels, even though we weigh the loss of the final output more.

Increasing the hidden states to the size of our best-performing RNN models shows significant improved performance despite evident overfitting, however this architecture is still inferior for test accuracy and exponentially larger model size.

Stacks	Params	Hidden Size	Dropout	Train Loss	Val Loss	Val CER	Test CER
2	6.0M	128	0.05	0.78	0.81	25.08	31.53
3	6.1M	128	0.05	1.50	1.39	26.94	30.54
3	23M	256	0.1	0.70,unstable	0.86	16.79	18.54

Table 4: Inception-GRU Results for 50 epochs, lr 1E-3

4 Discussion

4.1 LSTM

The 3-layer bidirectional LSTM achieved a final CER of **15.25%** with greedy CTC decoding after 50 epochs of training. The model improved significantly with dropout regularization, with a dropout of 0.4 reducing the loss gap; in the initial run, this indicated a high level of overfitting despite the RandomBandRotation and SpecAugment augmentations. This seems to help prevent co-adaptation, where nodes overly rely on other nodes, in the recurrent layers.

4.2 CNN+LSTM

For the CNN+LSTM model, the model was identified to not be sufficiently learning features during Run #1 due to high validation and test loss. Dropout regularization was applied to increase model robustness. The optimal dropout value was determined to be 0.1, as 0.2 led to a drop in performance. This model benefitted from dropout regularization, indicating that limited regularization was required to stabilize training. However, the fall in performance at increasing dropout to 0.2 indicates that aggressive dropout minimized active neurons to an extent that erased vital signals and, effectively, minimized the model’s capacity for feature learning.

4.3 GRU

The experimental results from the GRU models suggest several insights regarding model design for single-user EMG decoding. First, the strong impact of the augmentation pipeline indicates that improving generalization through data diversity may be more important than simply increasing model capacity. The consistent gap between validation and test performance in runs without augmentation highlights the role of augmentation as an effective regularization strategy.

Increasing hidden size beyond 256 did not lead to consistent improvements in performance. Larger GRU configurations occasionally achieved competitive validation accuracy but often produced worse test results, suggesting that additional capacity may increase the risk of overfitting without improving generalization in the single-user setting.

4.4 GRU-RAU

The GRU-RAU architecture produced generally competitive results across most configurations but did not consistently outperform the baseline GRU when comparing models with similar hidden sizes and layer counts. One possible explanation is that the GRU encoder already captures temporal dependencies effectively for this task, limiting the benefit of adding an additional attention mechanism.

Another possibility is that the RAU module requires further hyperparameter tuning or different architectural configurations to demonstrate stronger improvements. Future work could explore whether attention-based recurrent architectures such as RAU provide greater benefits in more complex scenarios, such as cross-user generalization, larger datasets, or tasks with greater variability in typing behavior.

4.5 Inception-GRU

The significant improvement in accuracy when increasing the number of hidden states suggests that the earlier models were too simple/not tuned well enough to adequately describe the data, despite their parameter count being similar to our most performant models. Perhaps the 3-stack architecture

performance could be improved by adding layer normalization after the subnetwork that auxiliary loss is calculated from to reduce instability when backpropagating through it twice.

Early experiments with this architecture showed that it can predict the general location of the pressed key, and mispredicted characters are often only a key off (e.g. “r” and “t”). The model seemed to require more context to resolve these ambiguous features, suggesting a need to increase the receptive field of the model. The similar performance over different depths contradicts this theory. Although we maintained proper test-validation data splits, perhaps the 3-stack models learned how to classify the short phrases of training and validation data well but could not generalize well to the sentences from the test data. The inception module is potentially even less suitable than standard convolution because it considers features across different scales of the input, simulating a fully-connected layer, which explains why only this architecture had worse test than validation accuracy.

4.6 Overview

The CNN+RNN model achieved better performance than the Baseline CNN. This indicates that, by the addition of the RNN to these hybrid model, the model performed its intended function of capturing long range dependencies with the RNN which were outside of the receptive field of the CNN. For the task of decoding sEMG signals to predict QWERTY keystroke values, expanding the architecture to capture temporal dependencies was critical as character strings are interdependent and knowledge of previous and future characters assists in the prediction of current keystrokes.

Testing both LSTM and GRU architectures allows comparison between two widely used recurrent models for sequential signal decoding. LSTMs include additional gating mechanisms that can better preserve long-term temporal dependencies, which may help capture extended patterns in sEMG signals. GRUs use a simpler gating structure with fewer parameters, often leading to faster training and reduced risk of overfitting on smaller datasets.

It was found that the LSTM architecture performed best under a high dropout threshold while the CNN+LSTM required a rather moderate dropout value of 0.1. This is likely due to the restriction of input features into the LSTM architecture in the CNN+LSTM model. For the LSTM model, the data includes the entire inputted sEMG dataset while the CNN+LSTM model is only training on feature vectors selected by the CNN model. Aggressive dropout for the CNN+LSTM model likely had a negative impact on training due to the already restricted nature of input data.

It was determined that model performance was optimized to train using only a RNN based architecture. The premise of the CNN+RNN models was to extract key features with the CNN architectures then account for temporal dependencies with RNN architectures in which the inputted data was the deep feature vectors outputted by the CNN. A theory as to why abandoning the CNN architecture helped performance is that the CNN architecture was overly restrictive in determining key features within the sEMG data. By limiting the amount of input data for the RNN architectures, the RNN’s could have been unable to isolate as many vital temporal dependencies due to a lack of data.

An alternate theory to why the CNN+RNN hybrids performed worse is that large receptive field is just not ideal for this use case. Each additional convolution layer computes features across larger portions of the input, increasing receptive field to analyze the entire input. Whereas the RNN networks focus only on local temporal features like individual words, deep CNN networks will analyze temporal features across all time, like entire sentences. For best generalization given the variability of language, perhaps the best architecture should learn to spell rather than formulate sentences, hence the CNN hybrids performed worse to their simpler RNN counterparts.

5 References

References

- Ieee xplore document 9252155. IEEE Xplore, 2020. URL <https://ieeexplore.ieee.org/abstract/document/9252155>. Accessed: 2026-03-13.
- Awni Hannun, Ann Lee, Qiantong Xu, and Ronan Collobert. Sequence-to-sequence speech recognition with time-depth separable convolutions. *arXiv preprint arXiv:1904.02619*, 2019. URL <https://arxiv.org/pdf/1904.02619>.

NVIDIA Developer. What is LSTM? Long Short-Term Memory. Online, 2024. URL <https://developer.nvidia.com/discover/lstm>.

Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. *arXiv preprint arXiv:1409.4842*, 2014. URL <https://arxiv.org/abs/1409.4842>.

Guoqiang Zhong, Guohua Yue, and Xiao Ling. Recurrent attention unit. *arXiv preprint arXiv:1810.12754*, 2018. URL <https://arxiv.org/abs/1810.12754>.